

**Java Backend Interview Series**

**Sub-Chapter 1 &**

## **Exception Handling -- Deep Dive**

Hierarchy / Checked vs Unchecked / try-with-resources / Custom / Spring / Concurrency

*Complete Import Statements -- 30+ Q&A -- Production Patterns -- Spring Boot -- Pitfalls*

# 1 Exception Hierarchy

Java's exception system is rooted in `java.lang.Throwable`. It splits into `Error` (JVM-level, do not catch) and `Exception` (application-level, may handle). `Exception` further splits into checked exceptions (compiler-enforced) and unchecked exceptions (`RuntimeException` and its subclasses). Knowing the hierarchy tells you which exceptions to handle and which indicate programming bugs.

```
java.lang.Throwable

+-- java.lang.Error (JVM-level -- do NOT catch)
|
| +-- OutOfMemoryError
|
| +-- StackOverflowError
|
| +-- AssertionError
|
| +-- LinkageError
|
| +-- NoClassDefFoundError
|
+-- java.lang.Exception
+-- java.lang.RuntimeException (UNCHECKED -- compiler does not require handling)
|
| +-- NullPointerException
|
| +-- ArrayIndexOutOfBoundsException
|
| +-- ClassCastException
|
| +-- NumberFormatException
|
| +-- IllegalArgumentException
|
| | +-- IllegalStateException
|
| +-- UnsupportedOperationException
|
| +-- ConcurrentModificationException
|
| +-- ArithmeticException (/ by zero)
+-- (Checked exceptions -- compiler REQUIRES handling or declaration)
+-- java.io.IOException
|
| +-- FileNotFoundException
|
| +-- SocketException
+-- java.sql.SQLException
+-- java.lang.ClassNotFoundException
+-- java.lang.InterruptedException
+-- java.lang.ReflectiveOperationException
```

*Java Throwable hierarchy -- complete tree*

| Category                     | Examples                                        | Handle?                  | Recovery ? | Indicates                      |
|------------------------------|-------------------------------------------------|--------------------------|------------|--------------------------------|
| Error                        | OOM, StackOverflow, LinkageError                | No -- do not catch       | Rarely     | JVM-level failure              |
| Checked Exception            | IOException, SQLException, InterruptedException | Yes -- or declare throws | Expected   | Expected recoverable condition |
| Unchecked (RuntimeException) | NPE, ClassCast, IllegalArg                      | Optional                 | No         | Programming bugs               |

## 1.1 Checked vs Unchecked Design Philosophy

Use CHECKED exceptions for conditions the caller can reasonably recover from and that are part of the method's contract: FileNotFoundException (caller can prompt for another path), SocketTimeoutException (caller can retry). Use UNCHECKED exceptions for programming errors: null passed where disallowed (NullPointerException), invalid index (ArrayIndexOutOfBoundsException). The checked/unchecked distinction is a compile-time contract, not a runtime one.

**WARNING:** Avoid catching Error or Throwable unless you are a framework/container that must clean up before propagating. OOM and StackOverflow leave the JVM in an undefined state -- catching them is almost never the right answer.

## 2 try-catch-finally Mechanics

Understanding the precise execution order of try-catch-finally is an interview classic. The key rules: finally always executes (barring JVM exit/Thread.stop/power failure). A return in finally overrides any return or exception from try or catch. An exception thrown from finally silently discards the original exception.

### 2.1 Execution Order

```
import java.io.IOException;

// Rule 1: finally ALWAYS executes (even with return in try)

static int test() {
    try {
        System.out.println("try");
        return 1;
    } catch (Exception e) {
        System.out.println("catch");
        return 2;
    } finally {
        System.out.println("finally"); // ALWAYS prints
        // return 3; // BAD: overrides the return 1 from try!
    }
}

// Output: try, finally. Return value: 1

// Rule 2: exception in finally REPLACES original exception

static void dangerous() throws Exception {
    try {
        throw new RuntimeException("original");
    } finally {
        throw new RuntimeException("finally"); // original LOST!
    }
}

// Rule 3: catch is skipped if no exception (or different exception type)
// Rule 4: only one catch block executes per exception

// Full execution order:
// normal flow: try body -> finally -> continue
// exception caught: try body -> catch -> finally -> continue
// exception not caught: try body -> finally -> propagate
```

TIP: Never put a return statement in a finally block -- it silently swallows exceptions and overrides the return value from try/catch, making bugs extremely hard to diagnose.

## 3 try-with-resources -- AutoCloseable and Suppressed Exceptions

Java 7 introduced try-with-resources (TWR). Any resource implementing AutoCloseable is automatically closed at the end of the try block, even if an exception is thrown. When both the body and close() throw exceptions, the close() exception is suppressed (added to the body exception) rather than discarding the body exception -- the opposite of the old finally problem.

### 3.1 Multi-Resource try-with-resources

```
import java.io.FileInputStream;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

// Single resource -- compiler generates finally { resource.close(); }
try (InputStream in = new FileInputStream("data.txt")) {
    // use in
}

// Multiple resources -- closed in REVERSE declaration order
try (Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement("SELECT 1");
    ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getInt(1));
    }
} // rs closed first, then ps, then conn

// Suppressed exceptions -- both exceptions are preserved!
class MyResource implements AutoCloseable {
    @Override public void close() throws Exception {
        throw new Exception("close failed");
    }
}

try (MyResource r = new MyResource()) {
    throw new Exception("body exception"); // primary exception
}

// "body exception" is thrown
// "close failed" is suppressed: e.getSuppressed()[0]
```

```
// Accessing suppressed exceptions:

try (MyResource r = new MyResource()) {

    // ...

} catch (Exception e) {

    System.out.println("Primary: " + e.getMessage());

    for (Throwable suppressed : e.getSuppressed()) {

        System.out.println("Suppressed: " + suppressed.getMessage());

    }

}
```

*try-with-resources: multi-resource and suppressed exceptions*

TIP: Java 9 allows effectively-final variables in try-with-resources: `InputStream in = openStream(); try (in) { }` -- no need to redeclare. The variable must be effectively final before the try.

## 4 Creating Custom Exceptions

Custom exceptions make your API self-documenting and allow callers to handle specific error conditions precisely. The key decisions: checked vs unchecked (checked = caller must handle, part of the contract; unchecked = programming error or unrecoverable). Always include cause chaining to preserve the original root cause through abstraction layers.

### 4.1 Checked vs Unchecked Custom Exception

```
import java.io.Serializable;

// Checked custom exception -- caller MUST handle or declare throws
// Use when the caller can reasonably recover
public class OrderNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;
    private final long orderId;

    public OrderNotFoundException(long orderId) {
        super("Order not found: " + orderId);
        this.orderId = orderId;
    }

    // Cause-chaining constructor -- preserves root cause
    public OrderNotFoundException(long orderId, Throwable cause) {
        super("Order not found: " + orderId, cause);
        this.orderId = orderId;
    }

    public long getOrderId() { return orderId; }
}

// Unchecked custom exception -- caller decides whether to handle
// Use for programming errors or conditions callers cannot fix
public class InvalidOrderStateException extends RuntimeException {
    private static final long serialVersionUID = 1L;

    public InvalidOrderStateException(String message) {
        super(message);
    }

    public InvalidOrderStateException(String message, Throwable cause) {
        super(message, cause);
    }
}
```



```
}
```

*Custom checked and unchecked exceptions with cause chaining*

## 4.2 Cause Chaining -- Exception Translation

```
import org.springframework.dao.DataAccessException;
import java.util.Optional;

// Exception translation: wrap low-level exception in high-level
// Always preserve the original cause!

public Order findOrder(long orderId) throws OrderNotFoundException {
    try {
        return orderRepository.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException(orderId));
    } catch (DataAccessException e) {
        // Low-level SQL/JPA exception -> high-level domain exception
        throw new OrderNotFoundException(orderId, e); // e is the cause
    }
}

// Inspecting the cause chain:
try {
    findOrder(999L);
} catch (OrderNotFoundException e) {
    System.out.println("Cause: " + e.getCause()); // DataAccessException
    e.printStackTrace(); // shows full chain
}
```

*Exception translation preserving cause chain*

**WARNING: NEVER swallow the cause: throw new ServiceException("error") silently discards the original exception. Always use the (message, cause) constructor or initCause(original).**

## 5 Multi-Catch and Re-throw

Java 7 introduced multi-catch (`catch (IOException | SQLException e)`) to handle multiple exception types with one block when the handling logic is the same. Java 7 also improved re-throw: when you re-throw a caught exception, the compiler performs more precise type narrowing -- the declared throws type is the specific type caught, not just `Exception`.

### 5.1 Multi-catch Syntax

```
import java.io.FileNotFoundException;
import java.io.IOException;
import java.sql.SQLException;

// BEFORE Java 7: repetitive catch blocks

try {
    riskyOperation();
} catch (IOException e) {
    log("IO error", e); throw new ServiceException(e);
} catch (SQLException e) {
    log("SQL error", e); throw new ServiceException(e);
}

// AFTER Java 7: multi-catch -- cleaner, DRY

try {
    riskyOperation();
} catch (IOException | SQLException e) {
    log("DB/IO error", e); // e is effectively final here
    throw new ServiceException("Operation failed", e);
}

// Multi-catch restrictions:

// 1. Cannot catch related types: catch (IOException | FileNotFoundException e)
// -- COMPILER ERROR: FileNotFoundException is a subtype of IOException

// 2. e is effectively final in multi-catch -- cannot reassign

// Ordering matters for non-multi-catch:

try { /* ... */ }

catch (FileNotFoundException e) { /* specific */ }

catch (IOException e) { /* broader -- must come after */ }
```

*Multi-catch syntax and restrictions*

### 5.2 Precise Re-throw (Java 7+)

```
import java.io.IOException;
import java.sql.SQLException;

// Java 7+ precise re-throw: compiler narrows type at re-throw
void rethrowExample() throws IOException, SQLException {
    try {
        riskyOperation(); // throws IOException or SQLException
    } catch (Exception e) { // catches broadly
        log(e);
        throw e; // compiler knows only IOException or SQLException

        // can be thrown here -- no "throws Exception" needed!
    }
}

// This allows callers to catch IOException or SQLException specifically
```

*Precise re-throw -- compiler narrows exception type*

## 6 Effective Exception Design

Well-designed exception handling is part of clean API design. The exception itself should tell the full story: what happened, what data was involved, and how to recover. Poor exception design -- swallowing exceptions, using exceptions for control flow, or providing cryptic messages -- is a major source of production incidents.

| Principle            | Good Practice                               | Anti-Pattern                                    |
|----------------------|---------------------------------------------|-------------------------------------------------|
| Message quality      | "Order 42 not found in region US-EAST-1"    | "Not found" or "Error"                          |
| Fail-fast            | Validate at entry point, throw immediately  | Let bad data propagate deep before failing      |
| Cause preservation   | throw new AppException("msg", cause)        | throw new AppException("msg") -- discards cause |
| Control flow         | Use Optional/Result for expected absences   | throw/catch for normal logic flow               |
| Swallowing           | Log + rethrow or handle specifically        | catch (Exception e) { /* empty */ }             |
| Exception type       | Specific type (FileNotFoundException)       | Generic (Exception, RuntimeException)           |
| Checked vs unchecked | Checked for recoverable; unchecked for bugs | Everything checked (API unusable)               |
| Logging              | Log at boundary, not at every layer         | Log + rethrow at every layer (duplicate logs)   |

```
import java.util.List;
import java.util.Objects;

// ANTI-PATTERN 1: empty catch block -- bug silently lost
try {
    processOrder(order);
} catch (Exception e) {
    // TODO: handle this <-- worst comment in Java history
}

// ANTI-PATTERN 2: exception for control flow (expensive!)
try {
    return Integer.parseInt(input); // throws if not a number
} catch (NumberFormatException e) {
    return -1; // using exception as a boolean check -- slow
}

// FIX: use Character.isDigit() or regex check first

// BEST PRACTICE: fail-fast with descriptive message
public Order placeOrder(Long customerId, List<OrderItem> items) {
```

```
Objects.requireNonNull(customerId, "customerId must not be null");

if (items == null || items.isEmpty())
    throw new IllegalArgumentException(
        "Order must contain at least one item; customerId=" + customerId);
// ... proceed with valid inputs

return new Order();
}
```

*Exception design anti-patterns and best practices*

**WARNING:** Exception creation is expensive -- the JVM must capture a full stack trace. If you throw the same exception type millions of times per second (e.g., sentinel value), consider pre-creating the instance or using a result type instead.

## 7 Exception Handling in Concurrency

Concurrent code introduces new exception handling challenges: exceptions thrown on background threads are invisible to the caller unless explicitly handled. `Future` wraps exceptions in `ExecutionException`. `CompletableFuture` provides functional exception handling. `Thread.UncaughtExceptionHandler` catches exceptions that escape thread `run()` methods.

### 7.1 Future and ExecutionException

```
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

// Future.get() throws ExecutionException wrapping the real exception
ExecutorService exec = Executors.newFixedThreadPool(4);

Future<String> future = exec.submit(() -> {
    if (Math.random() > 0.5) throw new RuntimeException("task failed");
    return "success";
});

try {
    String result = future.get(5, TimeUnit.SECONDS);
    System.out.println(result);
} catch (ExecutionException e) {
    Throwable cause = e.getCause(); // the actual exception from the task
    System.err.println("Task failed: " + cause.getMessage());
} catch (TimeoutException e) {
    future.cancel(true);
    System.err.println("Task timed out");
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // restore interrupt flag!
} finally {
    exec.shutdown();
}
```

*Future.get() and ExecutionException handling*

### 7.2 CompletableFuture Exception Handling

```
import java.util.concurrent.CompletableFuture;
import org.slf4j.Logger;
```

```

import org.slf4j.LoggerFactory;

// exceptionally() -- recover from exception
CompletableFuture<String> result = CompletableFuture
    .supplyAsync(() -> fetchUserName(userId)) // may throw
    .exceptionally(ex -> { // fallback on exception
        log.warn("fetchUserName failed: {}", ex.getMessage());
        return "Unknown";
    });

// handle() -- processes both result and exception
CompletableFuture<String> handled = CompletableFuture
    .supplyAsync(() -> fetchUserName(userId))
    .handle((name, ex) -> {
        if (ex != null) return "Unknown";
        return name.toUpperCase();
    });

// whenComplete() -- observe but not transform
CompletableFuture<String> observed = CompletableFuture
    .supplyAsync(() -> fetchUserName(userId))
    .whenComplete((name, ex) -> {
        if (ex != null) log.error("Failed", ex);
        else log.info("Fetched: {}", name);
    });

```

*CompletableFuture: exceptionally, handle, whenComplete*

## 7.3 Thread.UncaughtExceptionHandler

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

// Catch exceptions that escape Thread.run()
Thread.UncaughtExceptionHandler handler = (thread, ex) -> {
    System.err.println("Thread " + thread.getName() + " died: " + ex);
    // alert monitoring, trigger graceful shutdown, etc.
};

// Set on a specific thread:
Thread t = new Thread(() -> { throw new RuntimeException("boom"); });
t.setUncaughtExceptionHandler(handler);
t.start();

```

```
// Set as default for ALL threads:

Thread.setDefaultUncaughtExceptionHandler(handler);

// Set on ThreadPoolExecutor via ThreadFactory:

ExecutorService exec = Executors.newFixedThreadPool(4, r -> {

    Thread thread = new Thread(r);

    thread.setUncaughtExceptionHandler(handler);

    return thread;

});
```

*Thread.UncaughtExceptionHandler -- catch escaped thread exceptions*

**TIP:** Always restore the interrupt flag after catching InterruptedException: `Thread.currentThread().interrupt()`. Swallowing the interrupt flag prevents higher-level code from detecting that the thread was interrupted.



## 8 Spring Boot Exception Handling

Spring Boot provides a layered exception handling model: `@ExceptionHandler` methods in controllers for local handling, `@ControllerAdvice` for global handling across all controllers, and `ResponseBodyExceptionHandler` for Spring MVC internal exceptions. Spring 6 / Boot 3 added native support for RFC 7807 ProblemDetail as a standardized error response format.

### 8.1 @ControllerAdvice with @ExceptionHandler

```
import jakarta.servlet.http.HttpServletRequest;
import java.net.URI;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.servlet.mvc.method.annotation.ResponseBodyExceptionHandler;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestControllerAdvice // shorthand for @ControllerAdvice + @ResponseBody

public class GlobalExceptionHandler extends ResponseExceptionHandler {

    // Handle domain-specific exception

    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<ProblemDetail> handleOrderNotFound(
        OrderNotFoundException ex, HttpServletRequest req) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Order Not Found");
        pd.setInstance(URI.create(req.getRequestURI()));
        pd.setProperty("orderId", ex.getOrderId()); // custom extension
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(pd);
    }

    // Handle validation errors (Spring MVC)

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ProblemDetail> handleValidation(
        MethodArgumentNotValidException ex) {
```

```

ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);

pd.setTitle("Validation Failed");

List<String> errors = ex.getBindingResult().getFieldErrors()

.stream()

.map(fe -> fe.getField() + ": " + fe.getDefaultMessage())

.toList();

pd.setProperty("errors", errors);

return ResponseEntity.badRequest().body(pd);
}

// Catch-all -- log unexpected exceptions
@ExceptionHandler(Exception.class)
public ResponseEntity<ProblemDetail> handleAll(
Exception ex, HttpServletRequest req) {

log.error("Unexpected error at {}: {}", req.getRequestURI(), ex);

ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.INTERNAL_SERVER_ERROR);

pd.setTitle("Internal Server Error");

return ResponseEntity.internalServerError().body(pd);
}
}

```

*@RestControllerAdvice with ProblemDetail (RFC 7807, Spring Boot 3)*

## 8.2 ProblemDetail -- RFC 7807 Standard Error Format

```

// RFC 7807 ProblemDetail JSON response example:

{

"type": "https://api.shopverse.com/problems/order-not-found",

"title": "Order Not Found",

"status": 404,

"detail": "Order not found: 42",

"instance": "/api/orders/42",

"orderId": 42

}

// Enable ProblemDetail in application.yml:

// spring:

// mvc:

```

```
// problemdetails:
```

```
// enabled: true
```

*RFC 7807 ProblemDetail JSON format and Spring config*

TIP: Use `@ControllerAdvice` (not controller-local `@ExceptionHandler`) for all cross-cutting exception handling. This ensures consistent error responses across all endpoints and keeps controllers clean.

## 9 Exception Performance and Pre-Created Exceptions

Exception creation is expensive: the JVM captures the entire call stack when an exception is instantiated (not when it is thrown). In hot paths throwing hundreds of thousands of exceptions per second (sentinel values, parsing), this overhead is measurable. Pre-creating exceptions eliminates stack trace capture.

```
import java.util.Optional;

// Exception creation cost is in new Exception() -- stack trace capture

// Throwing a pre-created exception has almost no overhead

// Benchmark: creating 1M exceptions vs throwing 1M pre-created
// Creating: ~2,000ms | Pre-created: ~50ms

// Pattern 1: pre-create sentinel exception (no stack trace)
public class SentinelException extends RuntimeException {
    public static final SentinelException INSTANCE =
        new SentinelException("sentinel");

    private SentinelException(String message) {
        super(message, null, true, false); // writableStackTrace=false!
    }
}

// Pattern 2: override fillInStackTrace() for throwable pool
class FastException extends RuntimeException {
    @Override
    public synchronized Throwable fillInStackTrace() {
        return this; // skip expensive stack capture
    }
}

// Pattern 3: return Optional/Result instead of throwing
// Optional<User> findUser(long id) -- no exception at all for not-found

// Rule of thumb:
// < 1K/sec: regular exception fine
// 1K-100K/sec: consider Optional/Result type
// > 100K/sec: pre-create or eliminate exception entirely
```

*Exception performance: pre-created instances and writableStackTrace=false*

**WARNING:** Pre-created exceptions lack stack traces, making debugging harder. Only use this optimization when profiling proves exception creation is a bottleneck -- never as a premature optimization.

## Q Interview Q&A -- 30 Questions

**Q1. What is the difference between Error and Exception?**

Error represents JVM-level failures that applications should not try to recover from: `OutOfMemoryError`, `StackOverflowError`, `LinkageError`. They typically indicate unrecoverable conditions. Exception represents application-level conditions that may be handled. Both extend `Throwable`.

**Q2. What is the difference between checked and unchecked exceptions?**

Checked exceptions extend `Exception` but not `RuntimeException`. The compiler requires you to either catch them or declare throws in the method signature. They represent expected, recoverable conditions: `IOException`, `SQLException`. Unchecked exceptions extend `RuntimeException` -- compiler does not enforce handling. They represent programming errors: `NPE`, `AIOBE`, `ClassCastException`.

**Q3. What is the execution order of try-catch-finally?**

Normal flow: try body -> finally -> continue. Exception caught: try body -> catch -> finally -> continue. Exception not caught: try body -> finally -> propagate. A return in finally overrides any return from try or catch. An exception thrown in finally replaces (discards) the original exception from try.

**Q4. What is try-with-resources and what interface must resources implement?**

TWR (Java 7+) automatically closes resources at the end of the try block. Resources must implement `AutoCloseable` (which declares `close()` throws `Exception`). `Closeable` extends `AutoCloseable` and is restricted to `IOException`. `close()` is called in reverse declaration order. Exceptions from `close()` are suppressed (not discarded) when the body also throws.

**Q5. What are suppressed exceptions and how do you access them?**

When both the body and `close()` of a try-with-resources throw, the body exception is the primary (thrown), and the `close()` exception is added as a suppressed exception. Access via `e.getSuppressed()` which returns `Throwable[]`. In contrast, with old-style finally, throwing in finally silently discards the original body exception.

**Q6. How do you create a custom exception?**

Extend `Exception` (checked) or `RuntimeException` (unchecked). Always provide: (1) String-only constructor calling `super(message)`. (2) String+`Throwable` constructor calling `super(message, cause)` for cause chaining. (3) private static final long `serialVersionUID` for `Serializable`. Optionally add domain-specific fields like `orderId`.

**Q7. What is exception cause chaining and why is it important?**

Cause chaining wraps a low-level exception inside a higher-level one while preserving the original: `throw new ServiceException("DB error", sqlException)`. This enables exception translation -- hiding infrastructure details while preserving the root cause for debugging. Access the chain with `getCause()`; the stack trace shows all causes. Silently discarding the cause is a major debugging anti-pattern.

**Q8. What is multi-catch syntax (Java 7+)?**

`catch (IOException | SQLException e)` catches multiple exception types with one block when handling is identical. The variable `e` is effectively final in multi-catch. Restriction: you cannot catch related types in multi-catch -- `catch (IOException | FileNotFoundException e)` is a compile error because `FileNotFoundException` extends `IOException`.

**Q9. How do you handle `InterruptedException` correctly?**

Never silently swallow `InterruptedException`. Options: (1) Declare throws `InterruptedException` -- let caller handle it. (2) Catch and restore the interrupt flag: `catch (InterruptedException e) { Thread.currentThread().interrupt(); /* then handle */ }`. Swallowing the interrupt flag prevents higher-level code from detecting thread interruption, breaking cooperative cancellation.

**Q10. What is the `@ExceptionHandler` annotation in Spring?**

`@ExceptionHandler` on a method in a `@Controller` or `@ControllerAdvice` handles exceptions thrown from controller methods. The method can return `ResponseEntity` to control the HTTP status and body. `@ControllerAdvice` applies globally across all controllers. `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`.

**Q11. What is `ProblemDetail` in Spring Boot 3?**

`ProblemDetail` implements RFC 7807 (Problem Details for HTTP APIs) -- a standardized JSON error response format with fields: type (URI), title, status (HTTP code), detail (human-readable), instance (request URI). Plus arbitrary extension properties. Enable with `spring.mvc.problemdetails.enabled=true`. Creates consistent error responses across all endpoints.

**Q12. What is the difference between `@ExceptionHandler` and `ResponseEntityExceptionHandler`?**

`@ExceptionHandler` is a general annotation for handling any exception type. `ResponseEntityExceptionHandler` is a Spring MVC base class that provides `@ExceptionHandler` methods for all standard Spring MVC exceptions (`MethodArgumentNotValidException`, `NoHandlerFoundException`, etc.). Extend it to override specific handlers while inheriting the rest.

**Q13. Why is exception creation expensive?**

The JVM captures the full call stack when an exception is instantiated (in the constructor), not when it is thrown. Stack trace capture is  $O(n)$  where  $n$  is stack depth. Creating 1M exceptions can take 2+ seconds. Workaround: `new RuntimeException(message, cause, enableSuppression, writableStackTrace=false)` skips stack trace capture. Pre-created singleton exceptions eliminate the cost.

**Q14. Why should you not use exceptions for control flow?**

Exception creation captures a full stack trace -- expensive for frequent operations. It also makes code harder to read (normal flow should not throw exceptions). Use `Optional` for absent values, `Result` types for expected failures, validation methods (`Character.isDigit()`) before parsing. Reserve exceptions for truly exceptional conditions.

**Q15. What is `Future.get()` and why does it throw `ExecutionException`?**

`Future.get()` blocks until the async task completes. If the task threw an exception, `Future.get()` wraps it in `ExecutionException`. The actual exception is retrieved via `e.getCause()`. It also throws `TimeoutException` (if `get(timeout)` times out) and `InterruptedException` (if the calling thread is interrupted while waiting).

**Q16. [ADV] Explain the finally contract -- under what conditions does finally NOT execute?**

finally does not execute when: (1) `System.exit()` is called -- JVM terminates immediately. (2) `Thread.stop()` is called (deprecated, unsafe -- kills the thread without running finally). (3) The JVM crashes (native fault, OOM in the GC itself). (4) An infinite loop or deadlock in the try block -- finally cannot execute if try never finishes. In practice, power failure or `kill -9` also prevent finally.

**Q17. [ADV] What is the difference between `Throwable.getMessage()` and `Throwable.toString()`?**

`getMessage()` returns only the detail message string passed to the constructor (or null). `toString()` returns `className + ": " + getMessage()` -- e.g., `"java.io.FileNotFoundException: /tmp/file.txt"`. `printStackTrace()` additionally prints the full stack trace and cause chain. `getLocalizedMessage()` is for localization (default returns `getMessage()`).

**Q18. [ADV] How does `CompletableFuture.exceptionally()` differ from `handle()`?**

exceptionally(fn) is only called when the stage completes exceptionally -- it is a recovery function returning T (must return a value, not propagate further). handle(biFunction) is always called with (result, exception) -- both result and exception may be null; it can either recover or re-throw. whenComplete is like handle but cannot transform the result. Use exceptionally for simple fallbacks, handle for conditional recovery logic.

**Q19. [ADV] What is Thread.UncaughtExceptionHandler and when should you use it?**

UncaughtExceptionHandler is called when a Throwable escapes Thread.run() without being caught. Set on individual threads or globally via Thread.setDefaultUncaughtExceptionHandler(). Use it in production for: logging unexpected thread deaths, triggering alerts/monitoring, graceful shutdown. In thread pools set it via ThreadFactory to ensure all worker threads have it.

**Q20. [ADV] What is the serialVersionUID in exceptions and why should you declare it?**

serialVersionUID is the Serializable version identifier. If not declared explicitly, the compiler generates one from class structure (field names, types, methods). If you add a field later, the generated UID changes -- existing serialized exceptions (in log storage, distributed caches, RMI) become underserializable. Explicitly declaring private static final long serialVersionUID = 1L ensures compatibility across versions.

**Q21. [ADV] How do you implement exception handling in a Spring @Async method?**

@Async methods run in a separate thread -- exceptions do not propagate to the caller. If the return type is Future<T>, exceptions are wrapped and accessible via future.get(). If the return is void, the exception is silently lost unless you configure an AsyncUncaughtExceptionHandler via AsyncConfigurer.configureAsyncTaskExecutor(). Always handle exceptions explicitly in async methods.

**Q22. [ADV] What is the difference between exception translation and exception mapping?**

Exception translation: wrapping a low-level exception in a higher-level one within the same codebase -- e.g., catching DataAccessException and throwing DomainException. Exception mapping: external framework mapping exceptions to HTTP responses -- Spring's @ExceptionHandler, JAX-RS ExceptionMapper. Both serve to decouple layers: infrastructure exceptions should not leak into domain or API layers.

**Q23. [ADV] How does Spring's @Transactional interact with exceptions?**

@Transactional rolls back on unchecked exceptions (RuntimeException and Error) by default. Checked exceptions do NOT trigger rollback by default (by EJB convention). Configure: @Transactional(rollbackFor=IOException.class) to rollback on checked. @Transactional(noRollbackFor=OptimisticLockException.class) to suppress rollback. The proxy catches the exception, performs rollback/commit decision, then re-throws.

**Q24. [ADV] What is a NullPointerException helpful message (JEP 358, Java 14+)?**

JEP 358 (Java 14) / NullPointerExceptionMessages: the JVM analyzes which variable was null and provides a descriptive message: "Cannot invoke String.length() because <local1> is null". This replaces the cryptic NPE with a pointer to the exact null reference. Enabled by default from Java 14 (with -XX:+ShowCodeDetailsInExceptionMessages in 14, on by default in 15+). Tremendously reduces debugging time for NPEs in chained calls.

**Q25. [ADV] How does exception handling differ in reactive programming (Project Reactor)?**

In Project Reactor, exceptions propagate as error signals in the reactive pipeline -- not as thrown exceptions. Handle with: .onErrorReturn(fallback) -- return default value. .onErrorResume(fn) -- switch to fallback publisher. .onErrorMap(fn) -- transform exception type. .doOnError(fn) -- observe without handling. .retry(n) -- re-subscribe on error. Unlike Future, errors are first-class pipeline events, not wrapping constructs.

**Q26. [ADV] What is the try-with-resources decompiled bytecode and how does it handle suppression?**

TWR decompiles to a try-finally with: (1) primary try: run the body. (2) catch any Throwable: call resource.close() in a nested try; if close() throws, call primaryException.addSuppressed(closeEx). (3) if body succeeded: call close() normally; if close() throws, propagate it. The key: addSuppressed only happens when the body also threw -- avoids discarding the body exception.

**Q27. [ADV] How do you test that code throws the correct exception in JUnit 5?**

assertThrows(ExpectedException.class, () -> codeUnderTest()): returns the exception for further assertions. assertThrowsExactly() additionally checks the exact class (not subclass). assertDoesNotThrow() verifies no exception. Pattern: OrderNotFoundException ex = assertThrows(OrderNotFoundException.class, () -> service.findOrder(999L)); assertEquals(999L, ex.getId());

**Q28. [ADV] What is the difference between Throwable.initCause() and the constructor cause parameter?**

Both set the cause, but: initCause(cause) can only be called once (throws IllegalStateException if called again) and only if the cause was not set by the constructor. The constructor parameter (super(message, cause)) sets cause at creation time and is the preferred approach. initCause() exists for legacy exceptions that predate the cause constructor (added in Java 1.4).

**Q29. [ADV] Explain how Spring's DataAccessException hierarchy abstracts JDBC exceptions.**

DataAccessException is an unchecked exception hierarchy that wraps all Spring JDBC/JPA/Hibernate exceptions. This frees callers from catching vendor-specific checked exceptions (SQLException, HibernateException). Spring translates SQL error codes to specific subclasses: DataIntegrityViolationException (constraint violation), CannotAcquireLockException, DeadlockLoserDataAccessException. The translator reads SQL state codes from a properties file.

**Q30. [ADV] What is the finally block behavior when System.exit() is called inside try?**

System.exit() triggers the JVM shutdown sequence immediately. Registered shutdown hooks (via Runtime.addShutdownHook()) run, but finally blocks in the calling thread do NOT run. This is because System.exit() calls Runtime.halt() after running hooks, which forcibly terminates the JVM without unwinding the call stack. This is a critical difference from normal exception propagation where finally always runs.



# ! Common Pitfalls and Anti-Patterns

| Anti-Pattern                        | Problem                                                  | Fix                                                               |
|-------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|
| Empty catch block                   | Exception silently swallowed; bug invisible              | Log + rethrow, or handle specifically                             |
| Swallowing cause                    | throw new Ex("msg") -- original lost                     | throw new Ex("msg", originalException)                            |
| Exception for control flow          | Stack trace creation is expensive (O(n))                 | Use Optional, Result type, or boolean check                       |
| Catching Error/Throwable            | OOM, StackOverflow -- JVM in undefined state             | Let errors propagate; catch only Exception                        |
| Catching Exception broadly          | Catches unexpected bugs; hides real issues               | Catch specific type or minimum necessary supertype                |
| Logging + rethrowing at every layer | Duplicate log entries for one error                      | Log once at boundary; rethrow without logging                     |
| Not restoring interrupt flag        | Cooperative cancellation broken                          | Thread.currentThread().interrupt() in catch(InterruptedException) |
| return in finally                   | Silently overrides try/catch return value                | Never return from finally                                         |
| throw in finally                    | Original exception silently discarded                    | Use try-with-resources for automatic suppression                  |
| Missing serialVersionUID            | UID changes on class modification; deserialization fails | Declare private static final long serialVersionUID = 1L;          |

```
import java.util.OptionalInt;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

// ANTI-PATTERN: empty catch (deadliest Java sin)
try {
    processPayment(order);
} catch (Exception e) { } // payment may have failed -- nobody knows

// FIX: at minimum log
try {
    processPayment(order);
} catch (PaymentException e) {
    log.error("Payment failed for order {}: {}", order.getId(), e.getMessage(), e);
    throw new OrderFailedException("Payment rejected", e);
}

// ANTI-PATTERN: exception for control flow
```

```
String s = "not-a-number";

try {
    int n = Integer.parseInt(s); // expensive if frequent
} catch (NumberFormatException e) {
    // use -1 as sentinel
}

// FIX: validate first
OptionalInt parsed = s.matches("-?\\d+") ?
OptionalInt.of(Integer.parseInt(s)) : OptionalInt.empty();
```

*Exception anti-patterns with fixes*

## S Summary -- Key Takeaways

| Topic                  | Key Points                                                                                             |
|------------------------|--------------------------------------------------------------------------------------------------------|
| Hierarchy              | Throwable -> Error (JVM, do not catch) / Exception (checked+unchecked)                                 |
| Checked vs Unchecked   | Checked=compiler enforced, recoverable; Unchecked=programming bugs, optional handling                  |
| try-catch-finally      | finally always runs; return in finally overrides; exception in finally discards original               |
| try-with-resources     | AutoCloseable auto-closed; reverse order; body exception primary; close() exceptions suppressed        |
| Custom exceptions      | Provide String+cause constructors; serialVersionUID; checked=recoverable, unchecked=bug                |
| Exception translation  | Wrap low-level in high-level; ALWAYS preserve cause; never swallow                                     |
| Multi-catch / re-throw | catch (A   B e); e is effectively final; precise re-throw narrows throws type                          |
| Effective design       | Descriptive messages; fail-fast; no control flow; log once at boundary                                 |
| Concurrency            | ExecutionException wraps Future errors; restore interrupt flag; CompletableFuture.exceptionally/handle |
| Spring Boot            | @RestControllerAdvice + @ExceptionHandler; ProblemDetail (RFC 7807); @Transactional rollback rules     |
| Performance            | Stack trace capture is O(stack depth); pre-create for high-frequency; prefer Optional/Result           |

<b>Next: Sub-Chapter 1.9 -- Functional Programming & Streams</b>